

A3 · Answer Key and Demo Script

CS425 · curl, nc, and ping

INSTRUCTOR COPY

Graded pass/fail. Every transcript below is real output captured from `onyx.boisestate.edu` while this was written, so you can run any of it at the podium and get the same shape. Numbers will differ; the shapes will not.

Say this before you hand out the sheet

"Three weeks ago you learned to read `dig`. Today, three more tools, and then you have everything A4 needs. `curl` talks HTTP `nc` talks nothing, so you have to. `ping` is the odd one out and you will see why. Three rounds, one sheet, write down what you see." One minute, then paper.

WHAT IS ON THE BOX

THING	MEASURED ON ONYX
<code>curl</code>	7.76.1, OpenSSL 3.5.5, nghttp2
<code>nc</code>	nmap-ncat 7.92 at <code>/usr/bin/ncat</code> , not BSD netcat
<code>ping</code>	<code>iputils 20210202</code>
outbound 80 / 443	open
outbound ICMP	blocked , which is the whole of Round 3
probe	in <code>~/ .bashrc</code> since A2; if missing, paste the function from the A2 page at the prompt

ROUND 1 · CURL

Both halves of the conversation

```
$ curl -v http://example.com/
> GET / HTTP/1.1
> Host: example.com
< HTTP/1.1 200 OK
< Content-Type: text/html
```

SAY: > is what you sent, < is what came back. Four lines of text is the whole of an HTTP exchange. They will write this by hand in Round 2.

Where the time goes · this is the layer model, printed

```
$ curl -o /dev/null -s -w 'dns={time_namelookup} connect={time_connect} tls={time_appconnect} ttfb={time_starttransfer} total={time_total} code={http_code}\n' https://www.boisestate.edu/
dns=0.002637 connect=0.020129 tls=0.055797 ttfb=0.088997 total=0.144840 code=200

$ ... the same thing against http:// instead
dns=0.022634 connect=0.034200 tls=0.000000 ttfb=0.057717 total=0.057802 code=200
```

SAY: the answers to the three questions are all in these two lines. (1) Largest jump is usually `connect→tls` or `tls→ttfb`. (2) Between `connect` and `ttfb` sits the TLS handshake and then the server thinking about your request. (3) `tls` collapses to **EXACTLY ZERO** over plain HTTP, because there is no TLS handshake to measure. Cumulative, not per-phase: each number is time since the start. Subtract neighbors to get the cost of a phase.

Question 4. `probe` reports `connect` and `total` (plus the status code). A group that answers "dns and connect" read the variable names without running `declare -f`.

PASS BAR: all six numbers recorded, and the group said `tls` is the one that went to zero.

ROUND 2 · NC

Refused versus timed out, which is A4 in miniature

```
$ nc -vz onyx.boisestate.edu 22
Ncat: Connected to 132.178.227.11:22.
Ncat: 0 bytes sent, 0 bytes received in 0.01 seconds.
```

```
$ nc -vz onyx.boisestate.edu 9
Ncat: Connection refused.
```

```
$ nc -vz -w 3 example.com 8081
Ncat: Connection to 172.66.147.243 failed: TIMEOUT.
```

SAY: three outcomes, three meanings. Connected, refused (something answered "no"), and timeout (nothing answered at all). Those are A4 stations 1, 3 and 2 respectively, a week early and on a machine nobody deliberately broke.

HTTP by hand, and why the sleep is there

```
$ printf 'GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n' | nc -v example.com 80
Ncat: Connected to 172.66.147.243:80.
Ncat: 56 bytes sent, 0 bytes received in 0.06 seconds.      ← no reply!

$ { printf 'GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n'; sleep 3; } | nc example.com 80
HTTP/1.1 200 OK
Content-Type: text/html
Transfer-Encoding: chunked
Connection: close
```

SAY: nmap-ncat closes the connection the instant stdin runs out. The request left the machine, and nc hung up before the answer arrived. **SHOW THEM THE "56 BYTES SENT, 0 BYTES RECEIVED" LINE:** the tool told them exactly what happened, in numbers, and it is the sort of line people scroll past.

The `\r\n` question is the same rule SMTP uses, so this is P1 preparation. A server given bare `\n` may tolerate it or may hang; either way they cannot see the difference in a terminal, which is the lesson.

PASS BAR: an HTTP status line obtained without a browser, and the group can say what the two nc failure messages mean.

ROUND 3 · PING, AND THE CONTRADICTION

The best ninety seconds in the activity

```
$ ping -c 3 onyx.boisestate.edu
3 packets transmitted, 3 received, 0% packet loss
```

```
$ ping -c 3 example.com
3 packets transmitted, 0 received, 100% packet loss
```

... and in Round 1, twenty minutes earlier, from this same machine:
\$ curl -o /dev/null -s -w 'code=%{http_code}\n' http://example.com/
code=200

SAY: outbound ICMP is blocked from Onyx, so *every* host on the internet looks dead to ping from here, including one that served them a web page in the same period. Put both lines on the board together.

Answers: (1) ICMP is network layer, riding directly in IP, so it has no ports and no idea what a service is. (2) No: pinging a host says nothing about any program running on it. (3) A failed ping is consistent with a dead host, a filtered ICMP path, a filter at either end, or a middlebox in between. It rules almost nothing out, which is why "it doesn't ping" is not a diagnosis.

PASS BAR: both ping results recorded side by side, and the group names more than one cause for the failure.

Running the room

- **Round 1 twenty minutes, Round 2 twenty five, Round 3 ten.** If time is short, cut Round 3's switch table but keep the ping example.com demonstration; it is ninety seconds and it is the best thing here.
- **The Round 2 listener** must bind to 127.0.0.1 and must be killed. It is a shared department machine.
- **Bare \n** may be tolerated by the server. "Nothing changed" is still a pass; the point is they could not have told from the terminal.
- **Send them out with the worksheet graded and returned before A4**, which is the following week and reuses all of this.